Week 1: Security Assessment Report

Vulnerabilities Found

- **Cross-Site Scripting (XSS):**
  Successfully executed a stored XSS attack. The payload <script>alert('XSS');</script> was entered into the "Add/Edit Product" form and was reflected without sanitization, triggering the script execution in the browser.

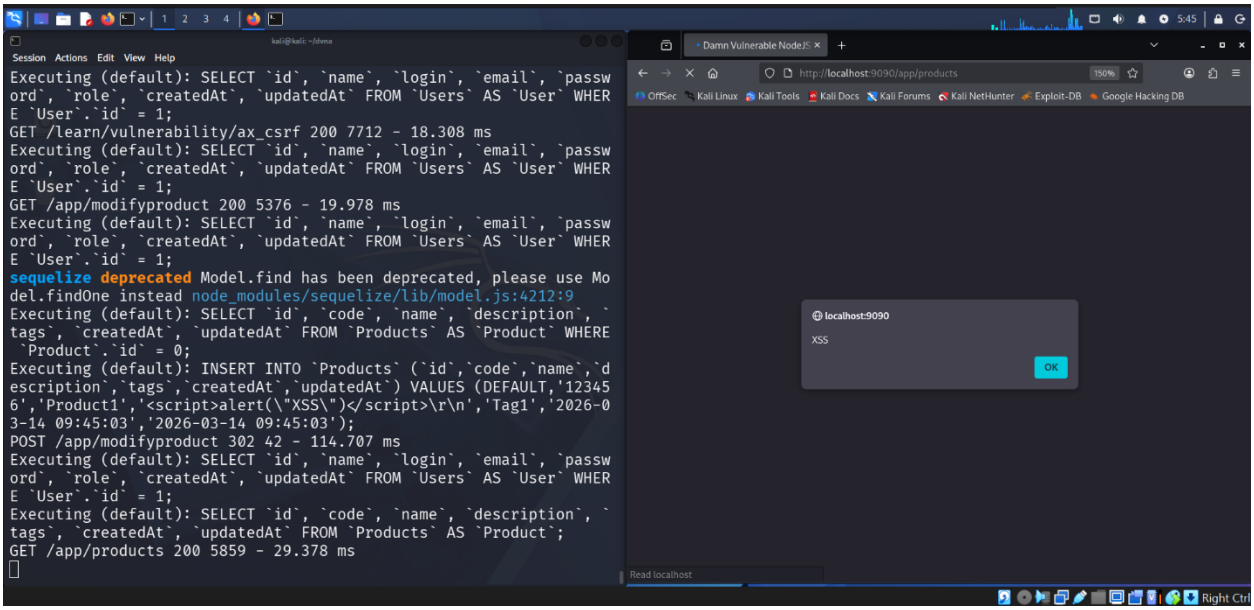

Figure 1: Successful XSS Payload insertion

- **Simple Security Misconfigurations:**
  Automated scanning with OWASP ZAP revealed a lack of basic HTTP security headers protecting the application's data transmission. Specifically, the application is missing the Anti-clickjacking (X-Frame-Options) and X-Content-Type-Options headers.

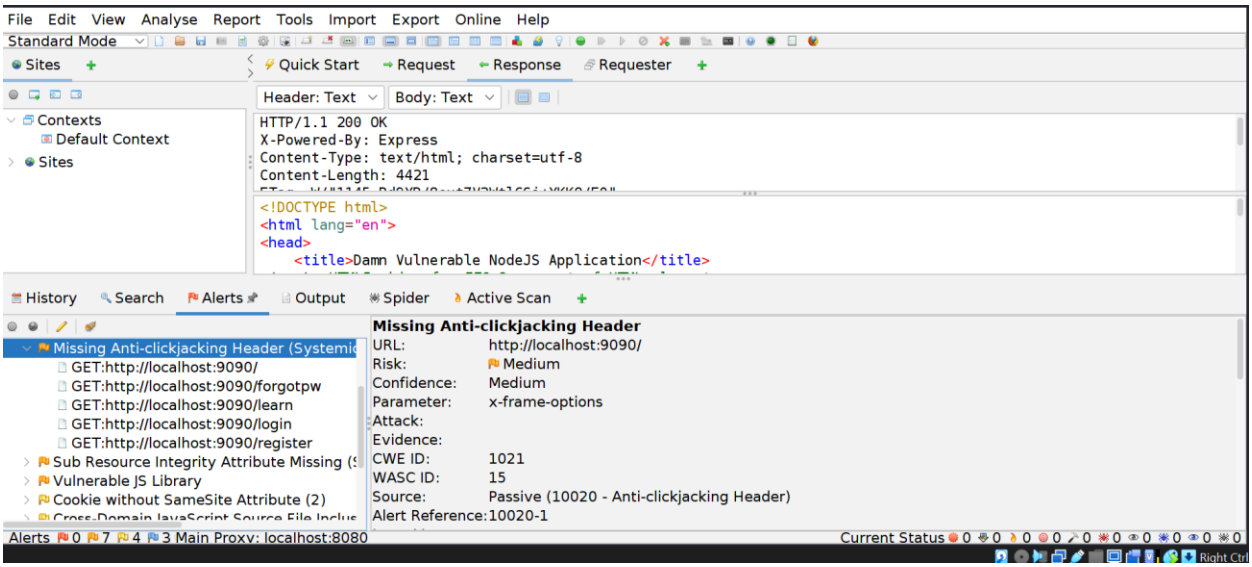

Figure 2:Missing Anti-clickjacking header.

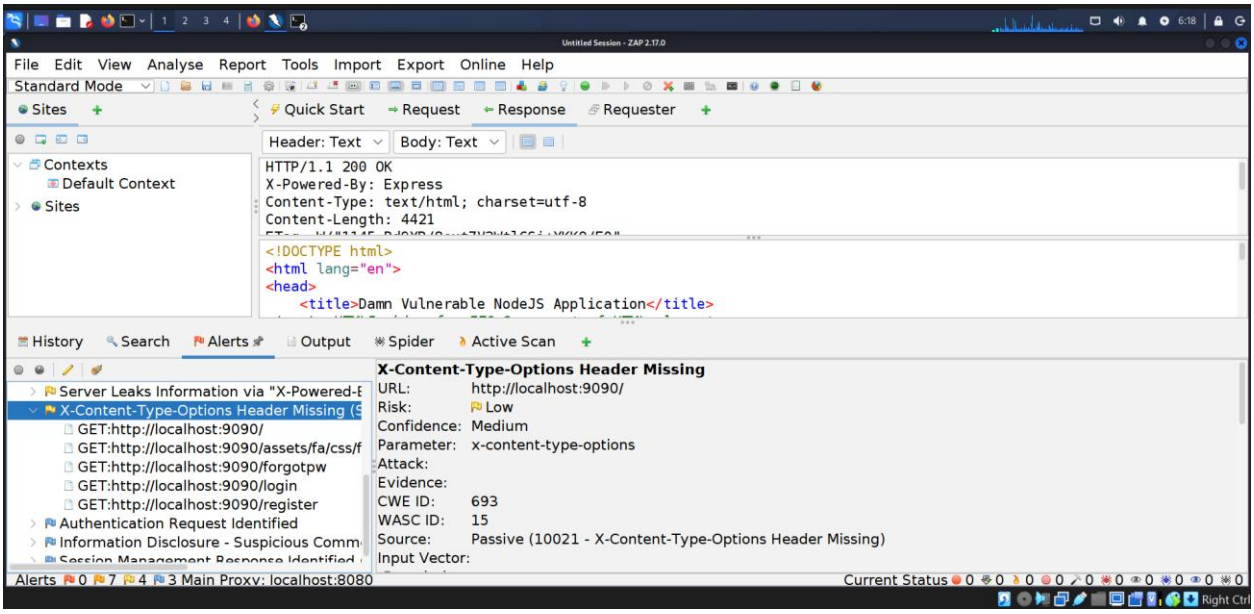

Figure 3: Missing X-Content-Type-Options header.

Vulnerabilities Tested (Mitigated by Application)

- **SQL Injection (Authentication bypass):**
  The login portal was tested for basic SQL injection using the payload admin' OR '1'='1. The attack was successfully neutralized by the application's Object Relational Mapper (Sequelize), which automatically escapes special characters.

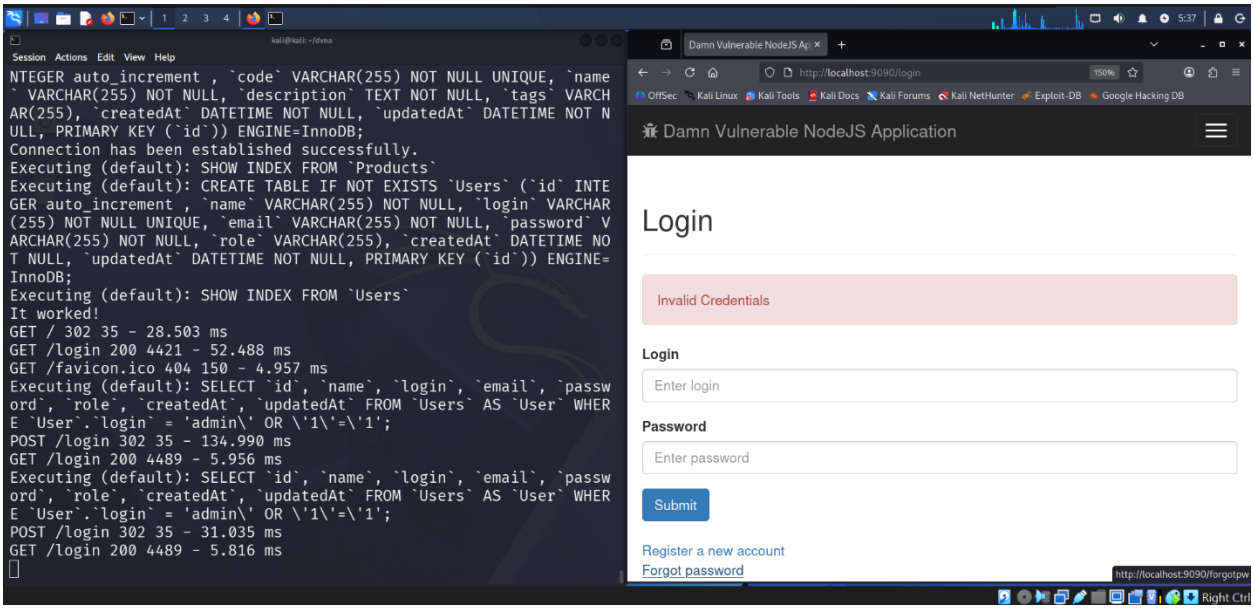

Figure 4: SQL Injection to bypass authentication failed.

- **Weak Password Storage:**
  Database inspection revealed that the application does *not* store passwords in plain text, mitigating weak password storage risks. User credentials are securely hashed utilizing the bcrypt algorithm.

Figure 5: Passwords Securely Hashed using the Bcrypt Algorithm.

Areas of Improvement

- **Sanitize and Validate Inputs:**
  Use the validator library to validate user inputs across all route handlers to neutralize malicious scripts.

- **Enhance Authentication:**
  Add basic token-based authentication using jsonwebtoken to manage user sessions securely.

- **Secure Data Transmission:**
  Use Helmet.js to secure HTTP headers and protect against the misconfigurations flagged by OWASP ZAP.

Week 2: Vulnerability Remediation Report

Overview

This report details the remediation steps taken to secure the Damn Vulnerable NodeJS Application (DVNA) based on the vulnerabilities identified during the Week 1 Security Assessment. Security libraries were integrated into the Express.js backend to enforce secure HTTP headers, sanitize user inputs, and modernize the authentication flow.

Securing HTTP Headers (Helmet.js)

- **Issue:** Automated scanning (OWASP ZAP) identified missing security headers, including Anti-clickjacking (X-Frame-Options) and X-Content-Type-Options.

- **Remediation:** Integrated the helmet middleware library into the main server.js entry point. Helmet automatically sets robust HTTP response headers to protect the application from common web vulnerabilities.

- **Verification:** A subsequent OWASP ZAP automated scan confirmed that the missing header alerts were successfully resolved.

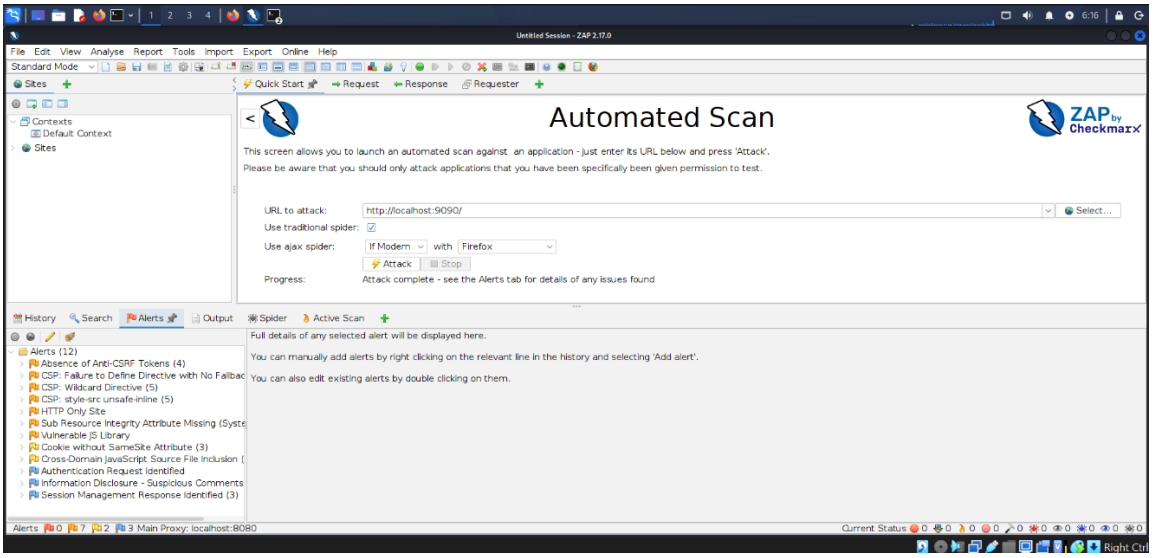

Figure 6: Missing Header Alerts Successfully Resolved using Helmet.js.

Mitigating Cross-Site Scripting (XSS)

- **Issue:** A Stored XSS vulnerability was present in the /modifyproduct backend route, allowing malicious scripts to be saved to the database and executed in the browser.

- **Remediation:** Installed the validator library and modified core/appHandler.js. The validator.escape() function was wrapped around all product input fields (name, description, code, and tags) to convert dangerous HTML characters (like < and >) into safe HTML entities prior to database insertion.

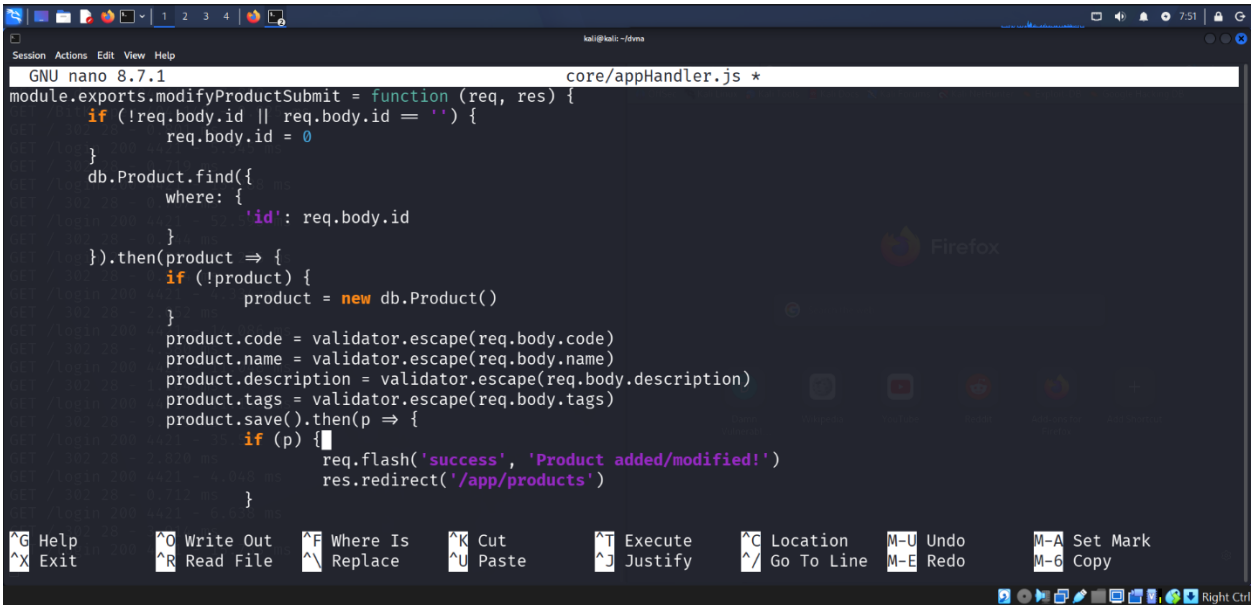

Figure 7: Product Input Field Wrapped inside validator.escape() Function.

- **Verification:** The payload <script>alert('XSS');</script> was submitted to the application. The input was successfully sanitized and rendered as safe text, neutralizing the attack.

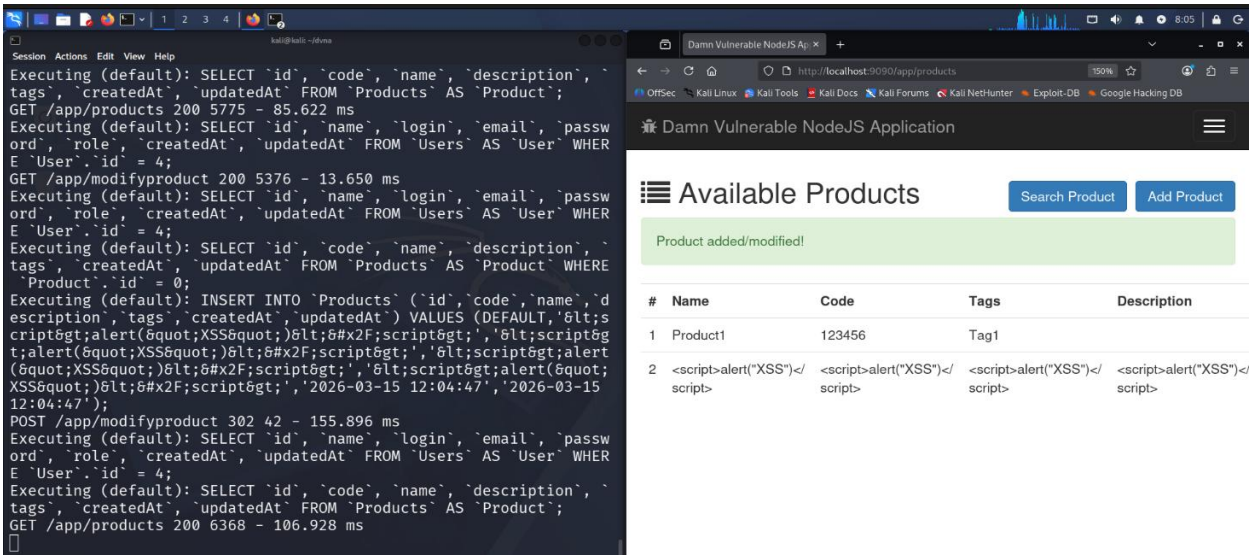

Figure 8: The Input Fields Successfully Sanitized.

Implementing Token-Based Authentication (JSON Web Tokens)

- **Issue:** The application relied on outdated, server-side session management for user authentication.

- **Remediation:** Replaced the legacy session architecture with Token-Based Authentication using the jsonwebtoken (JWT) and cookie-parser libraries.
    - Modified routes/main.js to cryptographically sign a JWT containing the user's details upon successful login and inject it into the client's cookies.
    - Rewrote the isAuthenticated middleware in core/authHandler.js to intercept protected route requests, extract the token, and verify its cryptographic signature before granting access.

- **Verification:** Successfully bypassed the legacy session checks and authenticated into the protected /learn dashboard using the newly generated JWT.

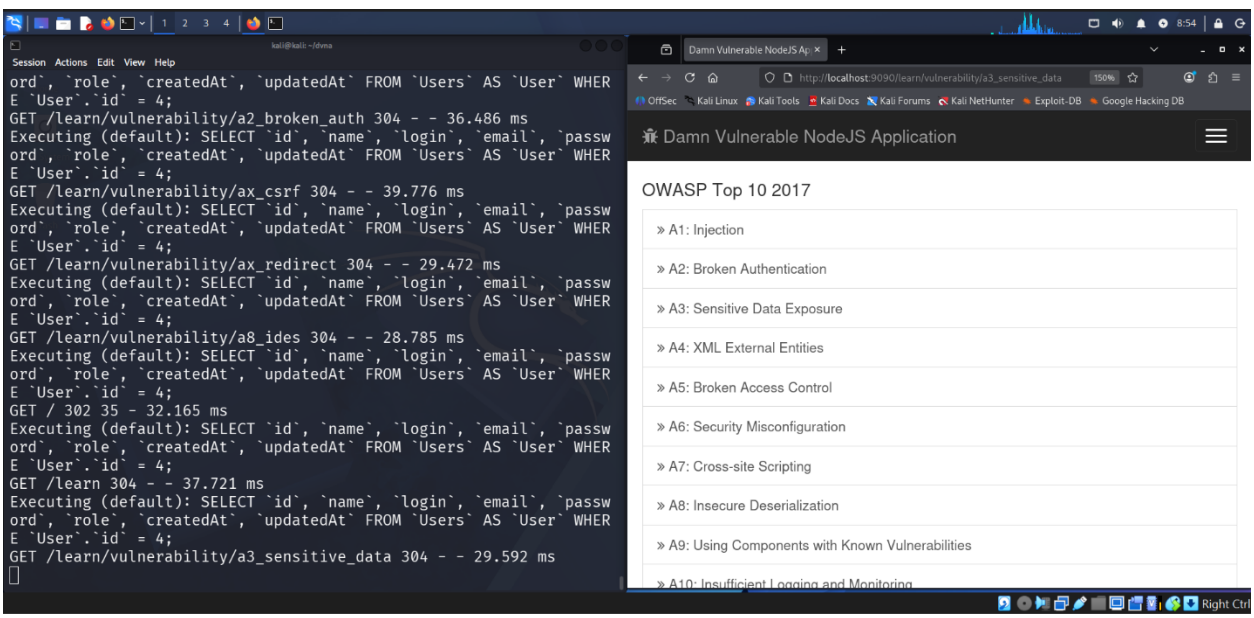

Figure 9: Successful Login and Token Generation.

Week 3: Advanced Security and Final Reporting

Basic Penetration Testing

- **Activity:** Conducted local network reconnaissance and service detection using Nmap against the host (127.0.0.1).

- **Findings:** Nmap successfully identified the open MariaDB database port (3306) and the Node.js application port (9090). Notably, the service fingerprinting on port 9090 returned the exact HTTP security headers implemented in Week 2 (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options), proving the active defense mechanisms are working against automated scanning enumeration.

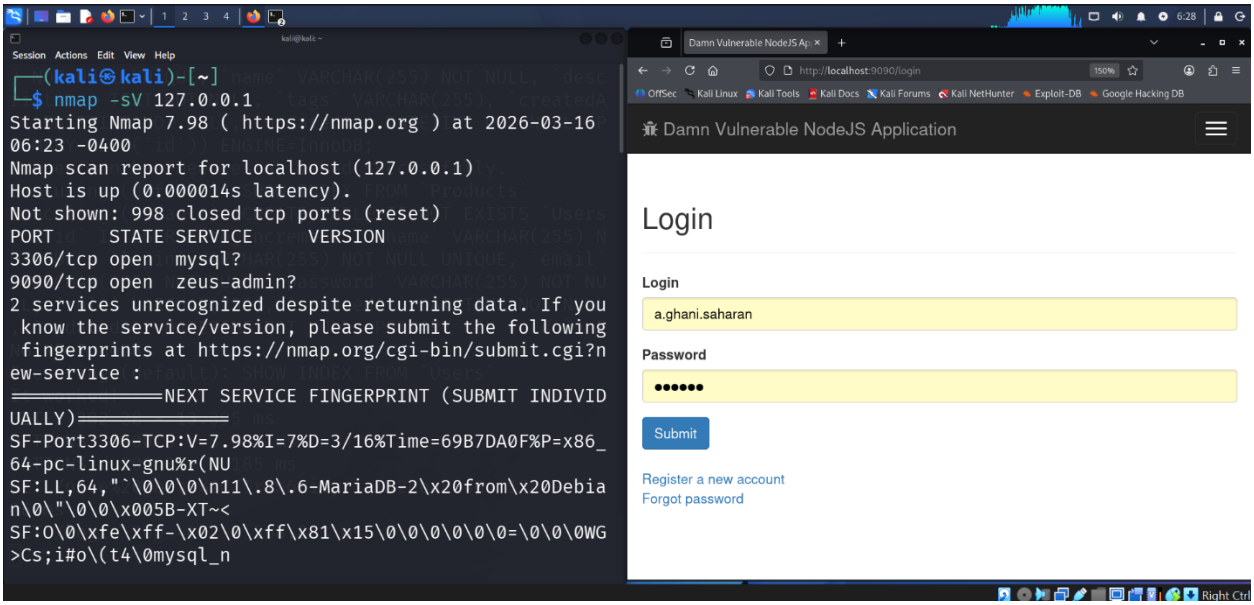

Figure 10: Defense Mechanisms Working Against Automated Scanning Enumeration

Application Logging Implementation

- **Activity:** Implemented application-level logging to monitor security and operational events.

- **Remediation:** Integrated the winston logging library into server.js. The logger was configured to output events to both the active console and a persistent security.log file.

- **Verification:** Successfully verified the logger's initialization by viewing the security.log file, which accurately captured the startup event.

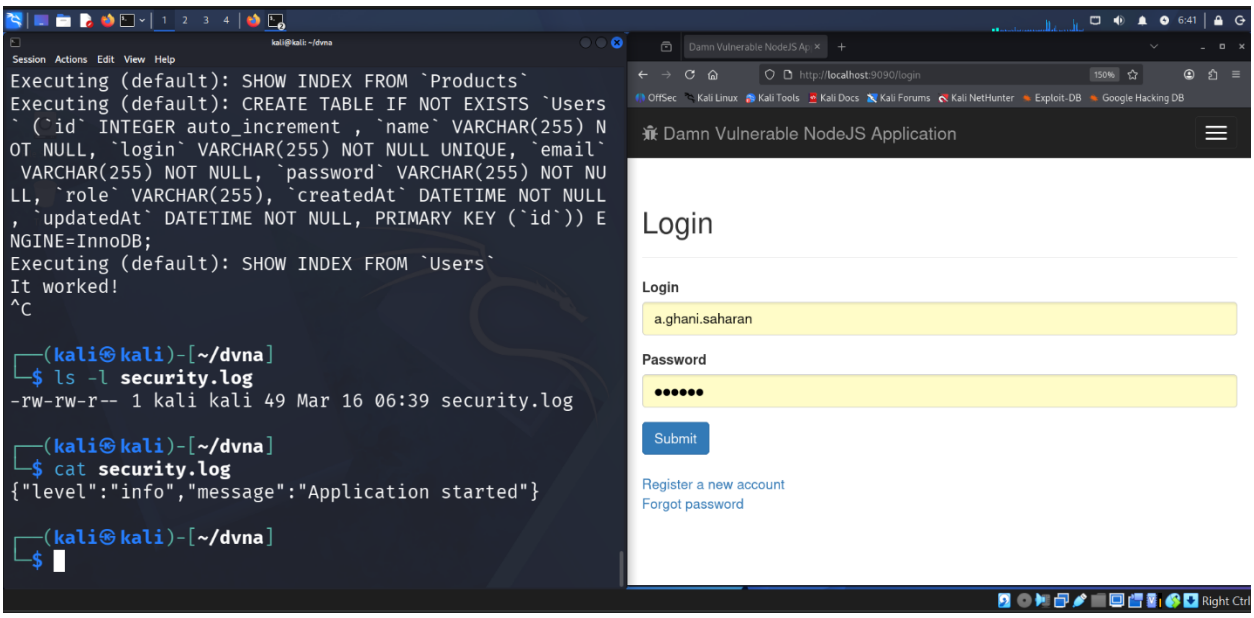

Figure 11: Verification of Logger's initialization

Simple Security Best Practices Checklist

To ensure the ongoing security of the application, the following baseline checklist was established:

- **Validate all inputs:** Ensure all user-supplied data is strictly sanitized and validated before processing or storing in the database (e.g., using validator) to prevent injection attacks.

- **Use HTTPS for Data Transmission:** Enforce TLS encryption for all traffic to protect sensitive data and session tokens from interception.

- **Hash and Salt Passwords:** Never store plaintext passwords; always utilize strong, modern hashing algorithms (like bcrypt) with randomized salts.

Week 4: Advanced Web Security Enhancements

API Security Hardening (Rate Limiting and CORS)

- **Issue:** The authentication endpoints were vulnerable to automated brute-force attacks, and the API lacked restrictions on cross-origin resource sharing (CORS), potentially allowing unauthorized domains to interact with the server.

- **Remediation:** Implemented express-rate-limit on the /login route, capping requests to 5 attempts per 15-minute window per IP address. Additionally, integrated the cors middleware to strictly limit API interactions to the local frontend (http://localhost:9090).

- **Verification:** Manual testing confirmed that exceeding the login attempt threshold resulted in the server successfully dropping the requests and returning a custom error message.

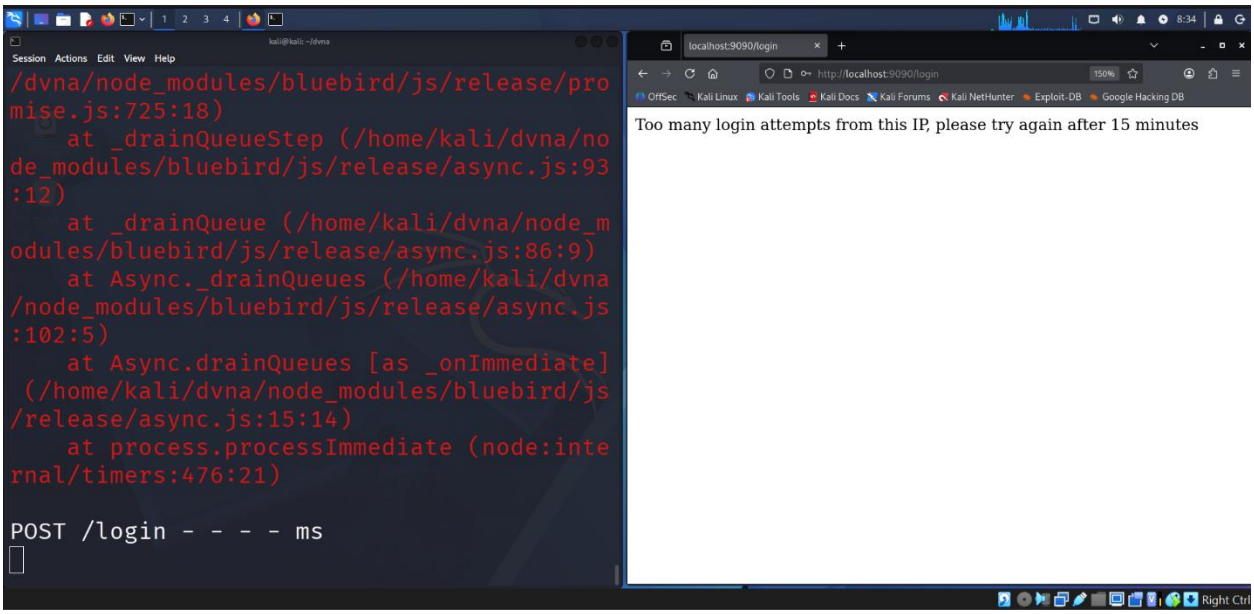

Figure 12: Rate Limiter successfully blocking brute-force login attempts.

Advanced Security Headers (CSP & HSTS)

- **Activity:** Upgraded the existing Helmet.js configuration to enforce a strict Content Security Policy (CSP) and HTTP Strict Transport Security (HSTS).

- **Remediation:** Configured CSP directives (defaultSrc, scriptSrc, styleSrc) to only accept resources from 'self', effectively neutralizing any injected malicious scripts that attempt to call out to external domains. Set HSTS to enforce HTTPS connections for a duration of one year.

- **Verification:** The strict CSP successfully blocked unauthorized external stylesheets, visually confirming the policy's active enforcement on the frontend.

Week 5: Ethical Hacking and Vulnerability Remediation

Automated SQL Injection Testing (SQLMap)

- **Activity:** Conducted an aggressive, automated penetration test against the /login endpoint using SQLMap to hunt for database vulnerabilities.

- **Findings:** SQLMap failed to identify any injectable parameters. The application's Object-Relational Mapper (Sequelize) inherently utilizes prepared statements to neutralize SQL syntax. Furthermore, the automated attack was actively intercepted and shut down by the newly implemented rate limiter, which dropped 145 malicious payloads with HTTP 429 (Too Many Requests) errors.

Cross-Site Request Forgery (CSRF) Protection

- **Issue:** Forms across the application were vulnerable to CSRF attacks, where a malicious actor could trick an authenticated user's browser into executing unwanted actions.

- **Remediation:** Integrated the csurf and cookie-parser middleware libraries. The server now generates a unique, cryptographically secure token for the session and requires that exact token to be submitted alongside any state-changing requests.

- **Verification:** Intercepted a valid login request using **Burp Suite**, manually tampered with the embedded CSRF token, and forwarded the modified payload. The server successfully identified the manipulation and rejected the request.

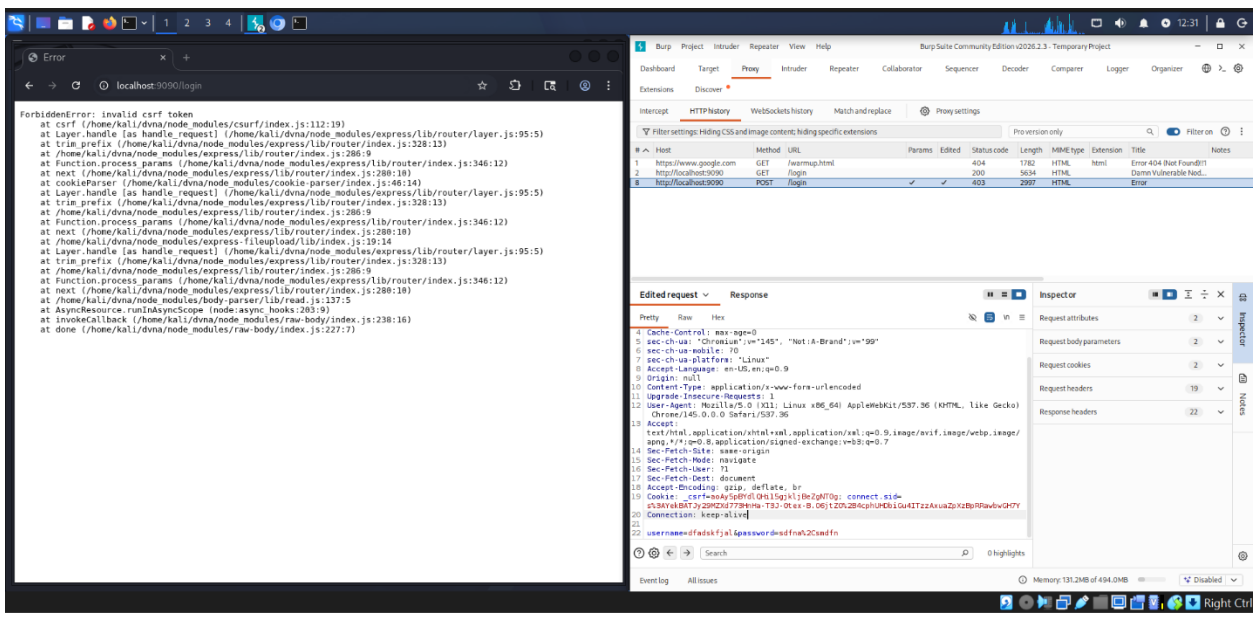

Figure 13: Burp Suite interception proving the active CSRF middleware defense.

Week 6: Advanced Security Audits and Deployment

Web Server Auditing (Nikto)

- **Activity:** Executed a comprehensive web server vulnerability scan using Nikto.

- **Findings:** The scan validated the active perimeter defenses, specifically logging the detection of the x-ratelimit headers and the strict access-control-allow-origin CORS configuration. Minor information disclosures were noted (e.g., X-Powered-By: Express), which have been logged for future hardening.

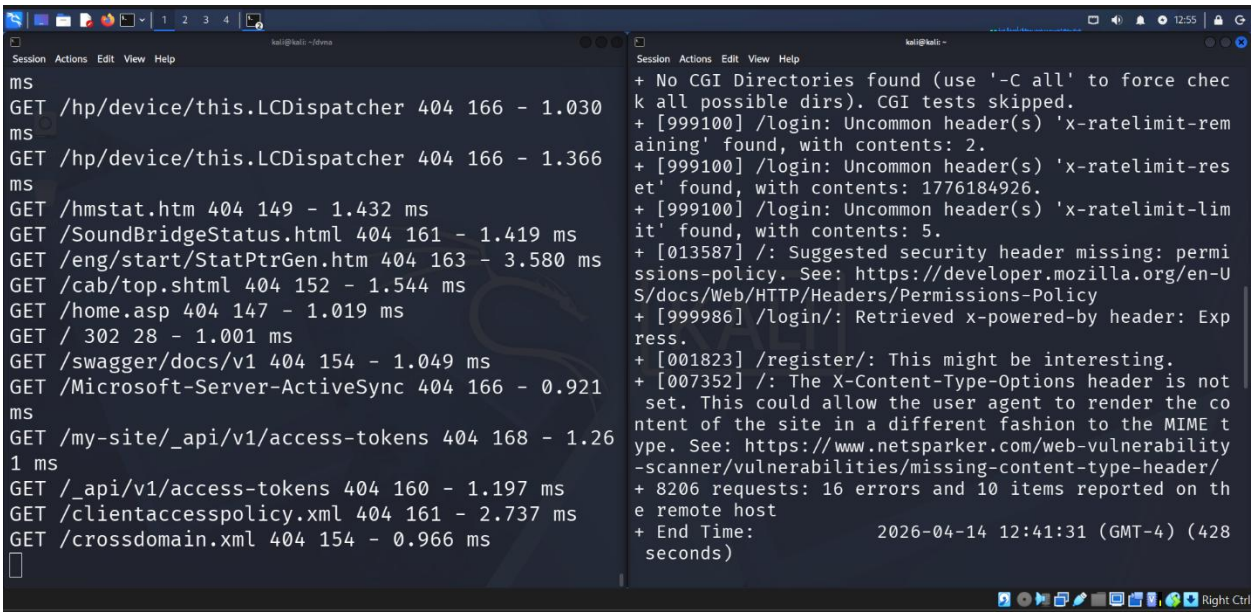

Figure 14: Nikto scan identifying the active rate-limiting security headers.

Host System Auditing (Lynis)

- **Activity:** Performed an internal system-level security audit on the Kali Linux host machine using Lynis.

- **Findings:** The host achieved a baseline Hardening Index of 60. The audit highlighted the absence of active host-level intrusion detection and firewall rules, providing a clear roadmap for infrastructure hardening prior to production deployment.

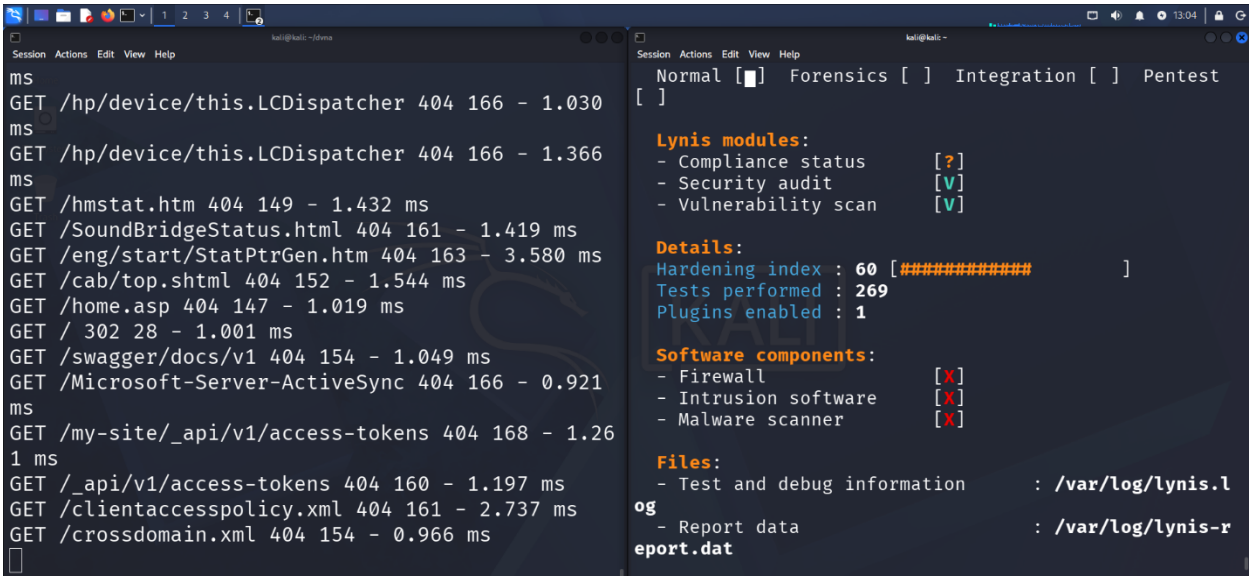

Figure 15: Lynis system audit highlighting host-level hardening requirements.

Dependency Auditing & Version Control

- **Activity:** Conducted a final security review of all third-party Node.js packages using npm audit to ensure no known vulnerabilities existed in the imported libraries.

- **Deployment:** All remediated source code, updated configurations, and security logs were successfully committed and pushed to the remote GitHub repository.

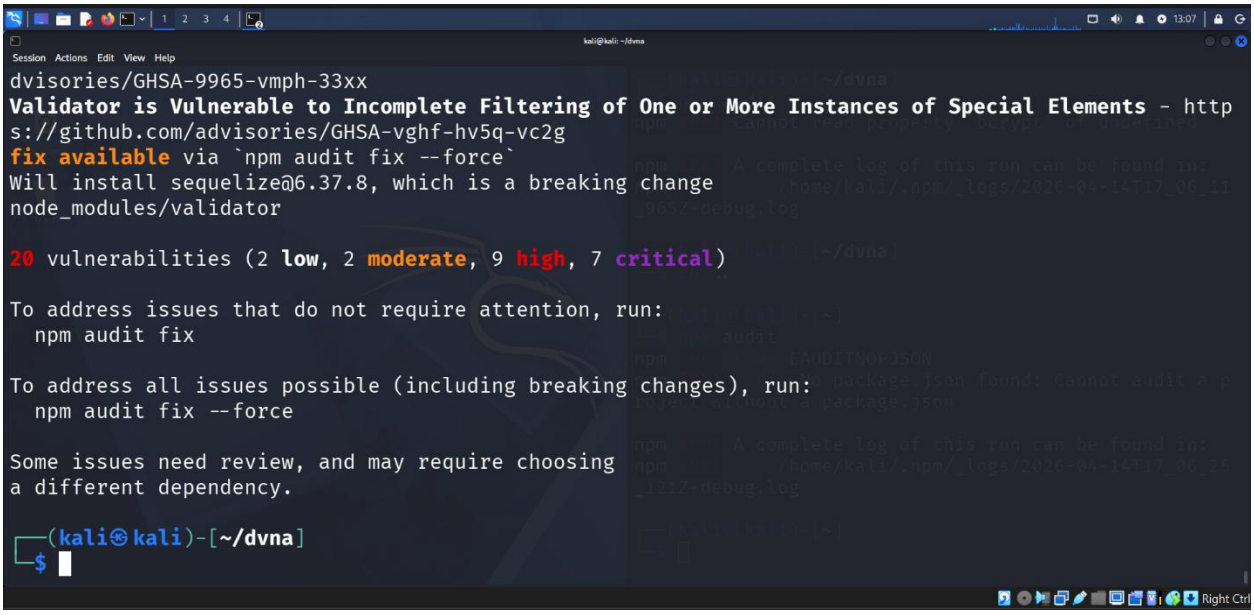

Figure 16: Final dependency security scan prior to deployment.

Future Recommendations

Based on the final audits, the following steps are recommended before deploying the application to a production environment:

1. **Infrastructure Hardening:** Configure the Uncomplicated Firewall (UFW) to restrict all non-essential ports and deploy an Intrusion Prevention System (IPS) such as Fail2Ban to monitor system logs for malicious activity.

2. **Header Optimization:** Remove the X-Powered-By header to prevent technology stack enumeration, and add the HttpOnly flag to the _csrf cookie to further protect it from client-side script access.